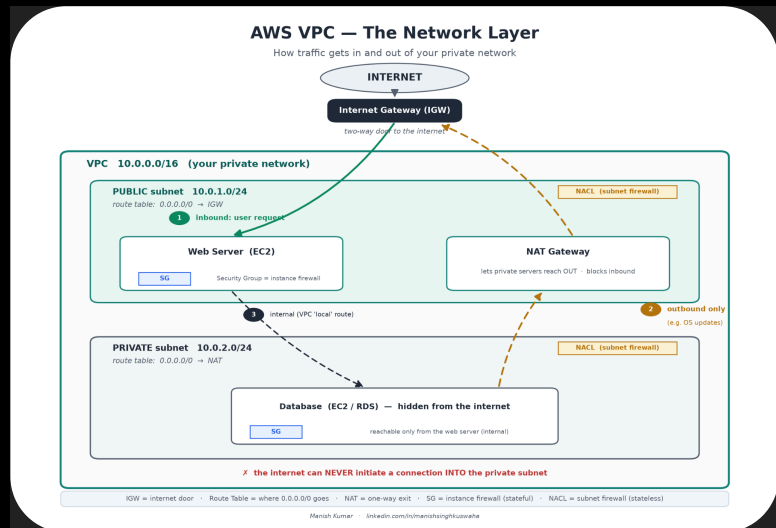


Friday 11:00 AM



what is the can you explain it in detail

This diagram is one of the **most common AWS VPC architectures** used in real-world applications. It shows how a secure web application is deployed on AWS.

I'll explain **every component** in simple language, step by step.

What is AWS VPC?

VPC (Virtual Private Cloud) is your own private network inside AWS.

Think of it like this:

- AWS = A huge city
- VPC = Your own apartment building inside that city

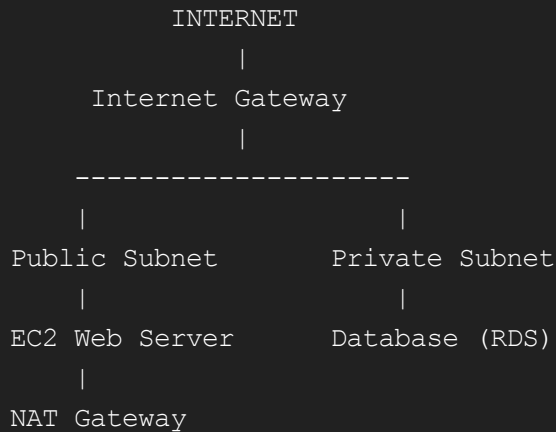
Only you control:

- IP addresses
- Servers
- Databases
- Routing

- Internet access
- Firewalls

Everything inside the VPC is isolated from other AWS customers.

The Complete Architecture



Let's understand every box.

1. Internet

This is the public internet.

Users

You
Me
Google
Facebook
Mobile App
Browser

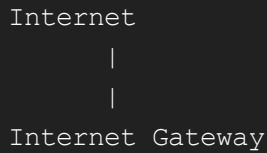
all live here.

Suppose someone types

www.amazon.com

The request starts from the Internet.

2. Internet Gateway (IGW)



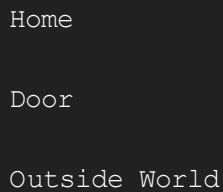
```
graph TD; Internet --> IGW[Internet Gateway]; IGW --> Internet;
```

The Internet Gateway is the **door** between your VPC and the Internet.

Without it,

No server inside your VPC can communicate with the Internet.

Think of it as



```
graph LR; Home --> Door; Door --> OutsideWorld[Outside World];
```

No door

↓

No one comes inside.

No one goes outside.

It supports two-way communication



```
graph TD; Internet1[Internet] --> EC21[EC2]; EC22[EC2] --> Internet2[Internet];
```

Both are allowed.

3. VPC

VPC

10.0.0.0/16

This is your private network.

The CIDR

10.0.0.0/16

means

Private IP range

10.0.0.1

↓

10.0.255.255

All machines inside the VPC get IPs from this range.

Example

Web Server

10.0.1.15

Database

10.0.2.20

Backend

10.0.3.25

All are inside the same private network.

4. Subnets

Inside a VPC, we divide the network into smaller sections called **subnets**.

Think of it like a college.

College

```
|  
|-- CSE Department  
|  
|-- ECE Department  
|  
|-- ME Department
```

Similarly

VPC

```
|  
|-- Public Subnet  
|  
|-- Private Subnet
```

Here

Public Subnet

10.0.1.0/24

Private Subnet

10.0.2.0/24

Each subnet has 256 IP addresses.

5. Public Subnet

A public subnet can communicate directly with the Internet.

How?

Because its route table contains

0.0.0.0/0

↓

Internet Gateway

Meaning

Any unknown destination

↓

Go to Internet

Servers placed here are usually

- Web Server
- Load Balancer
- Bastion Host

6. Route Table

Every subnet has a Route Table.

It decides

"Where should packets go?"

Example

Destination

10.0.0.0/16

↓

Local

means

Internal VPC communication.

Another entry

Destination

0.0.0.0/0

↓

Internet Gateway

means

Everything outside the VPC goes to the Internet.

Think of a Route Table like Google Maps.

You enter

Destination

↓

Google tells

Turn left

Turn right

Take Highway

The Route Table does the same for network traffic.

7. EC2 Web Server

Public Subnet

↓

EC2

This is your application server.

Examples

- Nginx
- Apache
- Flask
- Spring Boot
- Node.js

Suppose your website is

www.shop.com

The browser sends

Browser

↓

Internet

↓

Internet Gateway

↓

EC2

The EC2 processes the request and sends the response back.

8. Security Group (SG)

Notice the blue box labeled **SG**.

Security Group is a firewall for an individual EC2 instance.

Example rules

Inbound

Allow

HTTP

80

From Anywhere

Allow HTTPS

443

Allow SSH

22

Only My Laptop

Outbound

Allow Everything

If port 80 isn't allowed,
The website won't open.

Security Groups are **stateful**.

If a request enters,
The response is automatically allowed.

9. Network ACL (NACL)

Look at the orange box.

This is another firewall.

Difference

Security Group

Instance Level

Network ACL

Subnet Level

Imagine

Apartment Building

↓

Main Gate

↓

NACL

Apartment Door

↓

Security Group

So traffic must pass

Internet

↓

NACL

↓

Security Group

↓

EC2

NACL is **stateless**.

That means

If you allow inbound,

You must separately allow outbound.

10. Private Subnet

Now look at the lower part.

Private Subnet

This subnet has **no direct Internet access**.

There is **no route to the Internet Gateway**.

Even if someone knows the database IP,

They cannot reach it.

Example

Internet

↓

Database

✗ Blocked

Private subnet is used for

- Databases
- Internal APIs
- Redis
- Backend services

11. Database

RDS

or

EC2 Database

This stores

- Users
- Passwords (hashed)
- Orders
- Products
- Payments

Only the web server should connect.

Example

Web Server

↓

Database

Internet



Database

✗ Not Allowed

12. NAT Gateway

This is one of the most important AWS networking services.

A NAT Gateway lets **private subnet resources initiate outbound Internet connections**, while **blocking unsolicited inbound connections**.

Imagine the database server needs

```
yum update
```

```
apt update
```

```
Download Python
```

```
Download Packages
```

It needs Internet access.

But we don't want the Internet to connect back to the database.

So the path is

Database



NAT Gateway



Internet

Response

Internet



NAT



Database

But if someone tries

Internet



Database

It is blocked.

Why is the NAT Gateway placed in the public subnet?

Because it needs direct Internet access through the Internet Gateway to relay outbound traffic from private resources.

13. Local Route

The dashed arrow labeled

`internal (VPC local route)`

means

Resources inside the same VPC can communicate using private IP addresses.

Example

Web Server

10.0.1.20



Database

10.0.2.50

No Internet is involved.

Traffic stays inside AWS's private network.

Traffic Flow Explained

Scenario 1: User opens a website

```
graph TD
    Browser --> Internet
    Internet --> IGW[Internet Gateway]
    IGW --> PS[Public Subnet]
    PS --> WS[Web Server EC2]
    WS --> DB[(Database Private Subnet)]
    DB --> Response
    Response --> BB[Back to Browser]
```

This is the green path in the diagram.

Scenario 2: Database installs updates

```
graph TD
    DB[(Database)] --> NATG[NAT Gateway]
    NATG --> Internet
```

Response

```
graph TD
    Internet[Internet] --> NAT[NAT Gateway]
    NAT --> Database[Database]
```

The Internet **cannot start** a connection to the database.

Scenario 3: Web Server accesses Database

```
graph TD
    WebServer[Web Server] --> PrivateIP[Private IP]
    PrivateIP --> Database[Database]
```

This communication stays entirely within the VPC.

Why This Architecture Is Secure

- **Internet Gateway (IGW):** Provides two-way Internet connectivity for public resources.
 - **Public Subnet:** Hosts Internet-facing components like web servers and load balancers.
 - **Private Subnet:** Keeps sensitive resources like databases inaccessible from the Internet.
 - **NAT Gateway:** Allows private resources to download updates or packages without exposing them to inbound Internet traffic.
 - **Security Groups:** Stateful firewalls attached to individual instances.
 - **Network ACLs:** Stateless firewalls applied at the subnet boundary.
 - **Route Tables:** Determine where traffic is sent (locally within the VPC, to the Internet Gateway, or to the NAT Gateway).
-

Real-World Example: Amazon Shopping Website

Imagine you visit **amazon.com**:

1. Your browser sends a request over the Internet.

2. The request reaches the **Internet Gateway**.
3. The route table sends it to the **public subnet**.
4. The **EC2 web server** receives the request.
5. The web server queries the **database** in the private subnet for product information.
6. The database returns the data over the VPC's private network.
7. The web server builds the webpage and sends it back through the Internet Gateway to your browser.
8. Meanwhile, the database can still download operating system updates through the **NAT Gateway**, but no one on the Internet can connect directly to it.

This design—public web tier, private database tier, controlled routing, and layered firewalls—is one of the most common and recommended AWS VPC architectures for secure web applications.

Since you're learning AWS and MLOps, understanding this diagram is essential because the same pattern is used to deploy production ML APIs, web applications, and databases securely on AWS.

in the give diagram i want to know 1. what is NACL (Full form), why is it used , NAT getay and full flow of the diagram

Absolutely. This diagram represents a **standard 2-tier AWS VPC architecture** used in production. Let's understand **NACL**, **NAT Gateway**, and then the **complete flow** from start to finish.

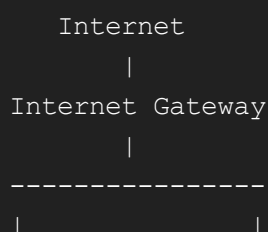
1. What is NACL?

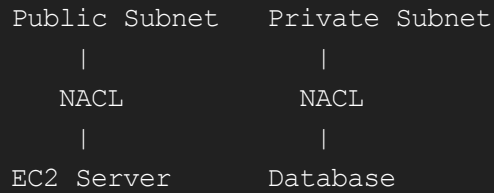
Full Form

NACL = Network Access Control List

It is a **subnet-level firewall**.

Think of it like this:





Notice that the **NACL protects the entire subnet**, not individual servers.

Why is NACL Used?

Suppose you have **100 EC2 instances** inside one subnet.

Without NACL:

```
EC2
EC2
EC2
EC2
EC2
...
```

You would need to configure firewall rules for every server.

Instead,



One NACL protects the entire subnet.

Example

Suppose you only want web traffic.

Allow

```
HTTP (80)
HTTPS (443)
```

Block

FTP (21)
Telnet (23)
RDP (3389)

Now every server inside that subnet automatically follows these rules.

NACL Rules

Unlike Security Groups,

NACL has two rule sets.

Inbound Rules

Internet
↓
Subnet

and

Outbound Rules

Subnet
↓
Internet

You configure both separately.

NACL is Stateless

This is the most important interview question.

Suppose a browser sends

Browser
↓
EC2

If inbound rule allows it

Allow Port 80

The request reaches EC2.

Now EC2 wants to reply.

EC2



Browser

The outbound rule must also allow it.

Otherwise,

The response is dropped.

That's why NACL is called **Stateless**.

Security Group vs NACL

Feature	Security Group	NACL
Full Form	Security Group	Network Access Control List
Works On	EC2 Instance	Entire Subnet
Stateful	✅ Yes	❌ No
Need Separate Outbound Rule?	No	Yes
Allow Rules	Yes	Yes
Deny Rules	No	Yes

2. What is NAT Gateway?

Full Form

NAT = Network Address Translation

A NAT Gateway lets **private subnet resources access the Internet**, while preventing the Internet from initiating connections to them.

Imagine your database is here.

Private Subnet

Database

The database needs to

Download

Python

Java

OS Updates

Security Patches

These packages come from the Internet.

Without NAT,

Database

↓

Internet

✗ Impossible

Because the private subnet has **no direct route to the Internet Gateway**.

With NAT Gateway

Database

↓

NAT Gateway

↓

Internet

Now it works.

Why is NAT Gateway placed in the Public Subnet?

Look carefully at your diagram.

Public Subnet

EC2

NAT Gateway

Why?

Because NAT itself needs Internet access.

Its route is

NAT Gateway

↓

Internet Gateway

↓

Internet

Then it forwards traffic on behalf of private instances.

Why Can't We Put NAT in the Private Subnet?

Suppose NAT is here.

Private Subnet

Database

NAT

Now NAT itself cannot reach the Internet.

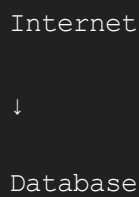
So it becomes useless.

That's why AWS requires a NAT Gateway to be placed in a **public subnet** with an Elastic IP.

Does NAT Allow Incoming Connections?

No.

Suppose a hacker tries



```
graph TD; Internet --> Database;
```

Internet

↓

Database

NAT blocks it.

Only requests that **originate from the private subnet** are allowed back.

Example



```
graph TD; Database1[Database] --> Arrow1[↓]; Arrow1 --> Google[Google]; Google --> Arrow2[↓]; Arrow2 --> Response[Response]; Response --> Arrow3[↓]; Arrow3 --> Database2[Database];
```

Database

↓

Google

↓

Response

↓

Database

This is allowed because the database initiated the connection.

3. Complete Flow of the Diagram

Now let's go through the entire architecture step by step.

Step 1: User Opens Website

Suppose you type

www.amazon.com

Traffic starts.

Your Browser

↓

Internet

Step 2: Internet Gateway

Traffic reaches

Internet Gateway (IGW)

Think of the IGW as the **main entrance** to your VPC.

Internet

↓

IGW

↓

VPC

Without an Internet Gateway, nothing inside the VPC can communicate with the public Internet.

Step 3: Route Table

The public subnet's route table contains

Destination

0.0.0.0/0

↓

Internet Gateway

This means all Internet traffic is routed through the IGW.

Step 4: NACL Check

Before reaching the server, traffic passes through the subnet's NACL.

Internet

↓

IGW

↓

Public NACL

↓

EC2

If the NACL blocks port 80 or 443, the packet is dropped.

Step 5: Security Group Check

After passing the NACL, traffic reaches the EC2 instance.

Before the EC2 accepts it, the Security Group checks its rules.

NACL

↓

Security Group

↓

EC2

If the Security Group allows HTTP/HTTPS, the request is accepted.

Step 6: EC2 Processes the Request

The EC2 web server (running Apache, Nginx, Node.js, Flask, Spring Boot, etc.) receives the request.

Example:

```
User requests
```

```
/products
```

The application now needs product data from the database.

Step 7: EC2 Connects to Database

The web server communicates using the VPC's internal network.

```
EC2
```

```
↓
```

```
Local Route
```

```
↓
```

```
Private Subnet
```

```
↓
```

```
Database
```

This traffic never leaves AWS.

It uses private IP addresses like

```
EC2
```

```
10.0.1.10
```

```
↓
```

```
Database
```

```
10.0.2.20
```

Step 8: Database Returns Data

The database sends the requested data back.

Database

↓

EC2

Step 9: EC2 Sends Response to User

The EC2 generates the webpage or API response.

EC2

↓

Security Group

↓

NACL

↓

Internet Gateway

↓

Internet

↓

Browser

The user now sees the website.

Separate Flow: Database Downloads Updates

Suppose the database needs a security update.

Database

↓

Private Route Table

The private subnet route table contains:

0.0.0.0/0

↓

NAT Gateway

So the path is:

Database

↓

NAT Gateway

↓

Internet Gateway

↓

Internet

The update is downloaded.

The response comes back through the same path:

Internet

↓

Internet Gateway

↓

NAT Gateway

↓

Database

Can the Internet Reach the Database?

No.

If someone tries:

Internet

↓

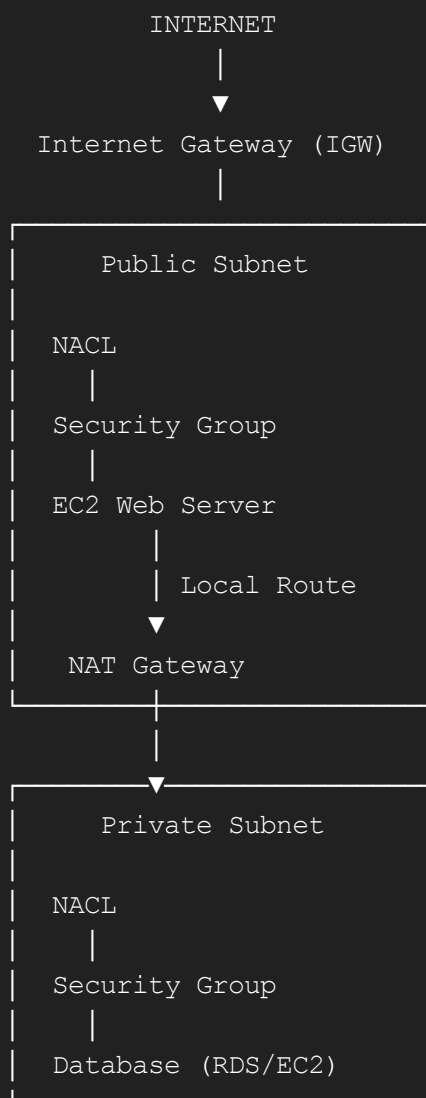
Internet Gateway

↓

Private Database

There is **no route** from the Internet Gateway to the private subnet, so the connection fails immediately.

Final Architecture Summary



Flow 1 (User Request):

Internet → IGW → Public NACL → Security Group → EC2 → Database → EC2 → IGW → Internet

Flow 2 (Database Update) :

Database → NAT Gateway → IGW → Internet → IGW → NAT Gateway → Database

Key interview points to remember

- **VPC** = Your private network in AWS.
- **Internet Gateway (IGW)** = Enables two-way Internet communication for public resources.
- **Public Subnet** = Has a route to the IGW.
- **Private Subnet** = No direct route to the IGW.
- **NAT Gateway** = Allows only outbound Internet access for private subnet resources.
- **NACL (Network Access Control List)** = Stateless firewall at the subnet level.
- **Security Group** = Stateful firewall attached to an EC2 instance or other AWS resource.
- **Route Table** = Determines where network traffic is sent.

This architecture is the foundation of many production AWS deployments, including web applications, REST APIs, and MLOps systems where a public API server communicates securely with private databases or ML services.

you explain only NACL, SG NAT gate way in detail this make me confuse if possible make a proper diagram for me

Yes. I think the previous explanation was too much at once. Let's forget the whole VPC architecture for now.

We'll understand only **three things**:

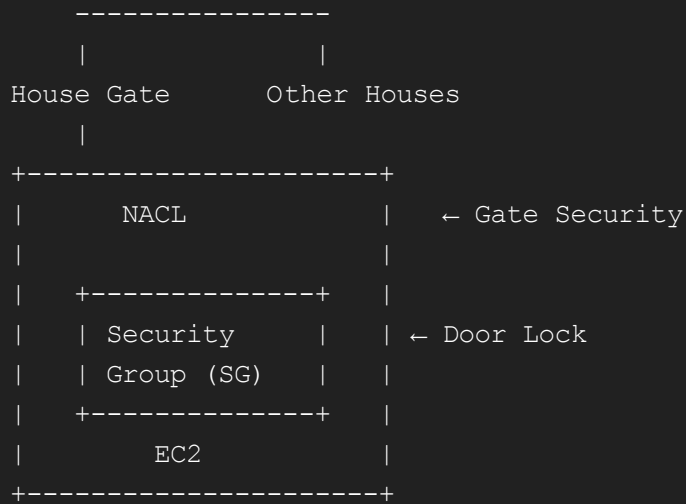
1. **NACL (Network Access Control List)**
2. **Security Group (SG)**
3. **NAT Gateway**

After you understand these three, the whole AWS VPC diagram will become very easy.

First Imagine This Simple House

INTERNET

|



Think of it like a house.

- **NACL = Security Guard at the Main Gate**
- **Security Group = Lock on Your Room Door**
- **EC2 = Your Room**

Let's understand each one.

1. NACL (Network Access Control List)

Full Form

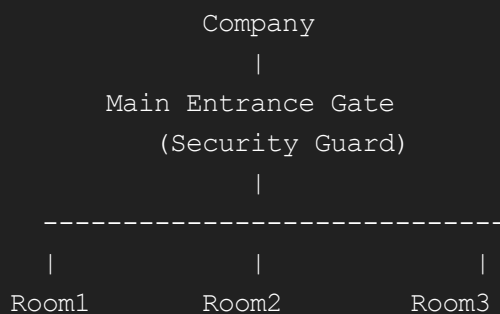
Network Access Control List

It is a **Subnet-Level Firewall**.

That means

It protects **every server inside a subnet**.

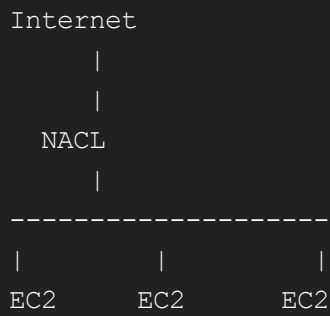
Imagine a company.



The guard checks everyone before allowing entry.

That guard is the **NACL**.

Where does NACL work?



Notice

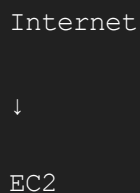
The NACL is **before every EC2**.

That means

Every request first reaches the NACL.

Example

Suppose a user sends

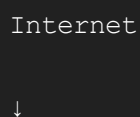


Before reaching EC2

NACL checks

```
Rule 100  
  
Allow Port 80  
  
YES
```

Then



NACL



EC2

Suppose someone tries SSH

Internet



EC2

NACL Rule

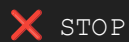
Block Port 22

Result

Internet



NACL



STOP

The EC2 never even sees the request.

Why do we use NACL?

Suppose your subnet contains

EC2

EC2

EC2

EC2

EC2

Instead of configuring firewall rules for every server,

You configure one NACL.

Now every EC2 follows those rules.

NACL is Stateless

This is the most important point.

Imagine this.

You allow

Internet

↓

EC2

Now EC2 wants to reply.

EC2

↓

Internet

Will AWS allow it automatically?

NO

You must also allow

Outbound Rule

So NACL needs

Inbound Rules

AND

Outbound Rules

Both are required.

This is called **Stateless**.

Easy Memory Trick

Think about a college gate.

Coming inside

↓

Guard checks ID.

Going outside

↓

Guard again checks ID.

Both directions are checked separately.

That is Stateless.

2. Security Group (SG)

Now suppose the user passed the NACL.

The request reaches the EC2.

But before entering EC2

Another firewall exists.

Internet

↓

NACL

↓

Security Group

↓

EC2

This is called the **Security Group**.

Full Form

There is no longer full form.

AWS officially calls it simply

Security Group (SG)

It works on

One EC2 instance.

Think of this.

Apartment Building

↓

Main Gate

↓

Your Flat Door

↓

Your Room

The building gate

↓

NACL

Your room door

↓

Security Group

Example

Your Security Group says

Allow

HTTP

Port 80

Now

Internet

↓

NACL

↓

Security Group

↓

EC2

Allowed.

Suppose someone tries

Port 22

Security Group says

NO

Result

Internet

↓

NACL

↓

Security Group

 STOP

Why Security Group?

Suppose

One subnet contains

Web Server

Database

Application Server

You don't want everyone to have the same rules.

Example

Web Server

Allow HTTP

Database

Allow only MySQL (3306)

Application Server

Allow only Port 8080

Each EC2 has its own Security Group.

Security Group is Stateful

This is another interview question.

Suppose

Internet

↓

EC2

Security Group allows Port 80.

EC2 sends a reply.

EC2



Internet

Does AWS automatically allow the reply?

YES

No outbound rule is needed for the response.

Because Security Group remembers

"This response belongs to an already allowed request."

This is called **Stateful**.

Easy Memory Trick

Think of your house.

A guest enters.

Your mother knows the guest.

When the guest leaves,

She doesn't check again.

Because she already remembers.

That is Stateful.

Difference Between NACL and Security Group

Feature	NACL	Security Group
Full Form	Network Access Control List	Security Group
Works On	Subnet	EC2 Instance
Position	Before EC2	Attached to EC2

Feature	NACL	Security Group
Stateful	✗ No	✓ Yes
Separate Inbound & Outbound Rules	Yes	No (responses are automatic)
Can Deny Rules?	✓ Yes	✗ No (only allow rules)

3. NAT Gateway

Now imagine another situation.

Your database is inside a private subnet.

Database

The database wants to install updates.

Database

↓

Ubuntu Update

↓

Internet

But

Private subnet cannot directly access the Internet.

So AWS introduces

Database

↓

NAT Gateway

↓

Internet

What is NAT?

Full Form

Network Address Translation

It acts like a **middleman**.

Imagine this.

Your father doesn't allow strangers to talk directly to you.

Instead

Everyone talks to your father.

Your father talks to you.

Internet

↓

Father

↓

You

Here

Father

↓

NAT Gateway

What does NAT Gateway do?

It allows

Private Server

↓

Internet

But blocks

Internet

↓

Private Server

Example

Database wants

Download Python

↓

Internet

Flow

Database

↓

NAT Gateway

↓

Internet

Internet replies

Internet

↓

NAT Gateway

↓

Database

Allowed.

Now a hacker tries

Internet

↓

Database

NAT says

✗ NO

Because

The database did not initiate this connection.

Why is NAT Gateway in the Public Subnet?

Because NAT itself needs Internet access.

```
Internet
  |
Internet Gateway
  |
Public Subnet
  |
NAT Gateway
  |
Private Subnet
  |
Database
```

If NAT were inside the private subnet, it couldn't reach the Internet to forward requests.

Easy Memory Trick

Imagine a company.

Employees (private servers) cannot leave the office directly.

They go through the receptionist.

Employee

↓

Receptionist

↓

Outside World

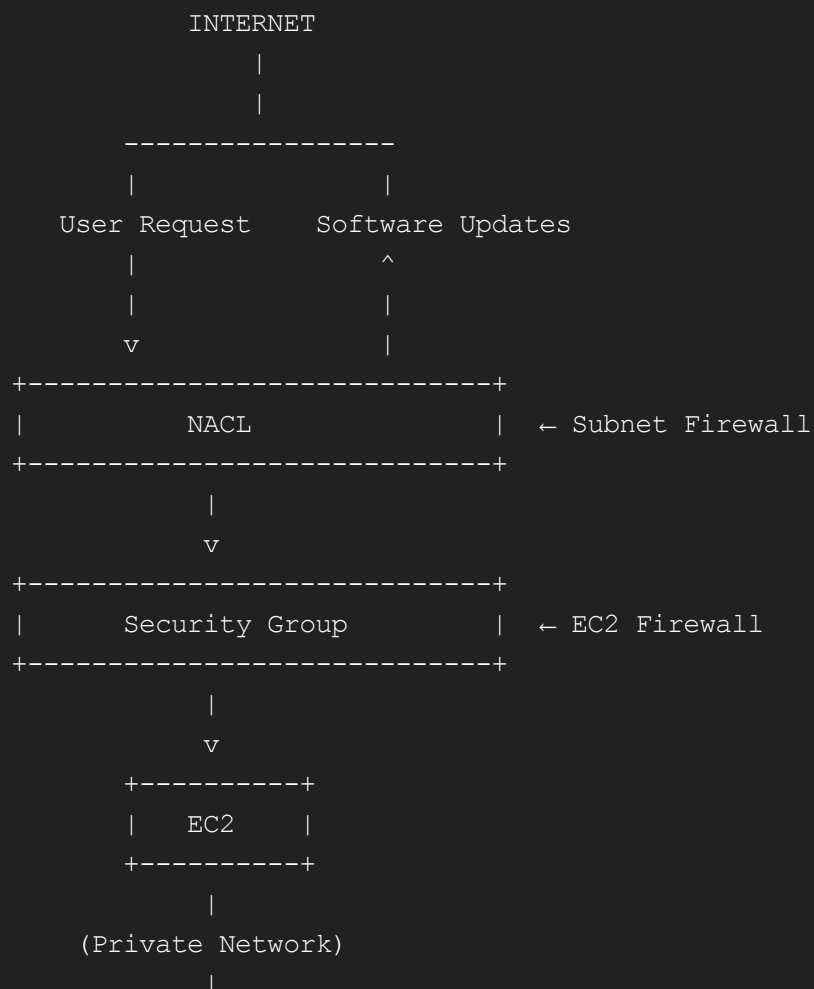
Visitors cannot walk directly to employees.

They must go through the receptionist, who blocks unauthorized access.

Here:

- **Employee** = Database in the private subnet
- **Receptionist** = NAT Gateway
- **Outside World** = Internet

Final Picture (Very Easy)





One-line summary to remember forever

- **NACL (Network Access Control List) → Protects the entire subnet** (like the main gate security guard). It is **stateless**.
- **Security Group (SG) → Protects a single EC2 instance** (like the lock on your room door). It is **stateful**.
- **NAT Gateway (Network Address Translation Gateway) → Allows private servers to access the Internet without allowing the Internet to directly access them** (like a receptionist or middleman).

If you're preparing for AWS interviews, these three services are among the most frequently asked networking concepts. Understanding the simple analogies above will help you remember both **where they are placed** and **why they exist**.

Friday 6:41 PM

IF I USE NACL SO WHAT IS LOGIC BEHIND THE USE OF SG

This is one of the **best AWS networking questions**. Many beginners think:

"If NACL already protects my subnet, why do I need a Security Group?"

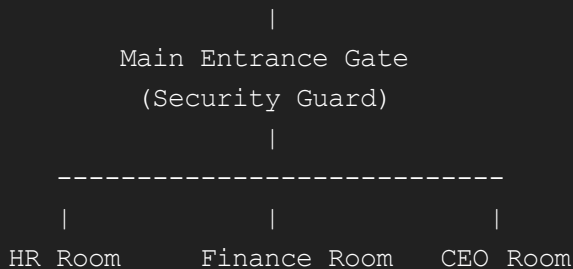
The answer is:

NACL and Security Group protect at two different levels.

Let's understand this with a real-world example.

Imagine a Company

COMPANY



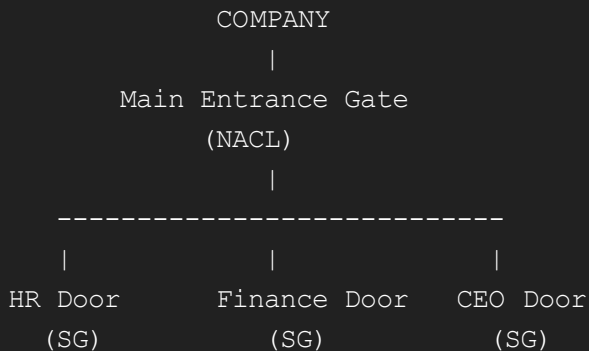
Now ask yourself:

Is only the **main gate security** enough?

No.

Because after entering the company, not everyone should enter every room.

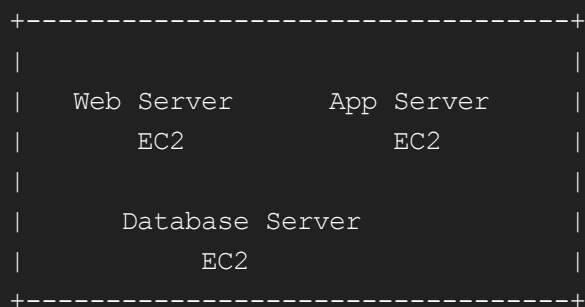
So every room also has its own lock.



AWS Works Exactly the Same Way

Suppose your public subnet contains three EC2 instances.

Public Subnet



The NACL protects the **entire subnet**.

But should all three servers have the same rules?

No!

Example

Suppose you have:

Web Server

Needs

```
HTTP  (80)
HTTPS (443)
```

because users visit the website.

Database Server

Needs only

```
MySQL (3306)
```

from the App Server.

It should **never** allow HTTP.

App Server

Needs

```
Port 8080
```

from the Web Server only.

If you only use NACL

The subnet rule would look like

```
Allow
```

```
80
```

443
3306
8080

Now **every EC2** inside that subnet can receive traffic on these ports.

That is **not secure**.

With Security Groups

Each EC2 gets its own firewall.

```
Public Subnet
+-----+

Web EC2
+-----+
| SG-1 |
+-----+
Allow:
80
443

App EC2
+-----+
| SG-2 |
+-----+
Allow:
8080

DB EC2
+-----+
| SG-3 |
+-----+
Allow:
3306 only

+-----+
```

Now every server has different permissions.

Think of It Like Airport Security

Layer 1

Airport Gate

Check Passport

↓

Enter Airport

This is

NACL

Layer 2

Boarding Gate

Check Boarding Pass

↓

Enter Plane

This is

Security Group

Even after entering the airport,
you cannot board every plane.

Another Example

Suppose a hacker reaches your subnet.

Internet

↓

NACL

↓

Subnet

NACL allows HTTP.

Now the hacker tries to connect to the database.

Database

↓

Security Group

↓

✗ DENIED

The Security Group blocks it.

Without the Security Group,

the database might also accept the connection if the subnet-level rules allowed it.

Why AWS Uses Both?

Because AWS follows the concept of **Defense in Depth** (multiple layers of security).

Instead of relying on one firewall,

AWS uses two.

Internet

|
|

+-----+

| NACL | ← First security layer

+-----+

|
|

+-----+

| Security | ← Second security layer

| Group |

+-----+

|
|

EC2

If one layer is misconfigured,

the other can still provide protection.

Real Example

Suppose someone accidentally changes the NACL.

They allow all traffic.

```
NACL
```

```
Allow Everything
```

Normally this is dangerous.

But your Security Group still says

```
Allow only
```

```
HTTP
```

```
HTTPS
```

The EC2 remains protected.

Now imagine the opposite.

Someone accidentally makes the Security Group too open.

Your NACL can still block unwanted traffic at the subnet boundary.

Easy Interview Answer

If an interviewer asks:

Why do we need a Security Group if we already have a NACL?

You can answer:

NACL protects the subnet, while Security Groups protect individual EC2 instances. NACL applies common rules to all resources in a subnet, but different EC2 instances usually require different access rules. Using both provides layered security (Defense in Depth), where NACL acts as the first line of defense and Security Groups provide fine-grained, instance-level access control.

One-line Memory Trick

NACL → Security at the Building Gate 🚪

Security Group → Security at Each Room Door 🚪

Building Gate (NACL):

- Everyone entering the building is checked.

Room Door (Security Group):

- Even after entering the building, you still need permission to enter a specific room.

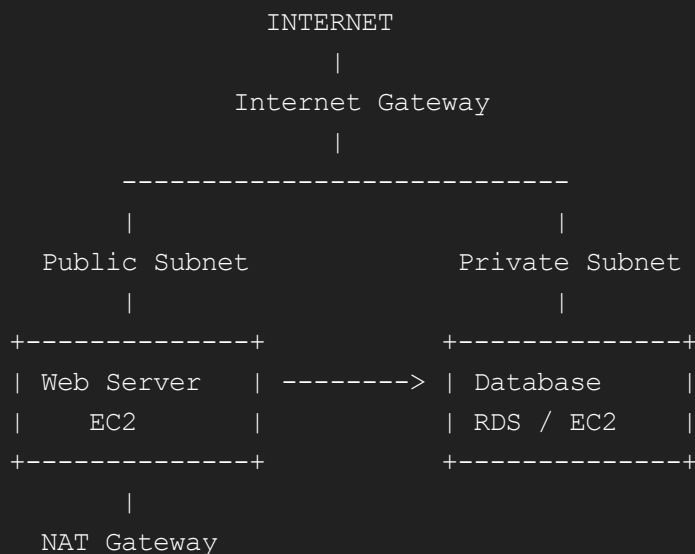
This is exactly why AWS uses **both** NACL and Security Groups instead of only one.

CAN YOU PROVIDE ME THE STEP BY STEP
IMPLEMENTATION PROCESS

Absolutely. Instead of memorizing AWS concepts, let's **build the architecture step by step**. This is exactly how a DevOps or Cloud Engineer would create it in AWS.

We'll create the architecture shown in your diagram from **scratch**.

Final Architecture We Want



Step 1: Create a VPC

First, create your own private network.

AWS Console

↓

VPC

↓

Create VPC

Example

Name : MyVPC

CIDR

10.0.0.0/16

Now AWS creates

MyVPC

10.0.0.0/16

Think of it as buying land.

Step 2: Create Two Subnets

Inside the VPC create

Public Subnet

10.0.1.0/24

and

Private Subnet

10.0.2.0/24

Now your VPC looks like

VPC

10.0.0.0/16

Internet Gateway

Associate this Route Table with

Public Subnet

Now

Public Subnet

↓

Internet

is possible.

Step 5: Launch EC2 in Public Subnet

Launch an EC2 instance.

Choose

Subnet

↓

Public Subnet

Enable

Auto Assign Public IP

YES

Now

Internet

↓

EC2

can happen.

Step 6: Create Security Group for Web Server

Create Security Group

Example

WebServer-SG

Inbound Rules

HTTP

80

Anywhere

HTTPS

443

Anywhere

SSH

22

My IP Only

Attach this Security Group to EC2.

Now

Internet

↓

Security Group

↓

EC2

Step 7: Create NACL for Public Subnet

Create

Public-NACL

Inbound

80

Allow

443

Allow

22

Allow

Outbound

All Traffic

Allow

Associate it with

Public Subnet

Now

Internet

↓

NACL

↓

Security Group

↓

EC2

Step 8: Create Private Route Table

Now create another Route Table.

This Route Table belongs to

Private Subnet

Initially

Destination

10.0.0.0/16

↓

Local

Only.

Notice

No Internet route.

Step 9: Launch Database

Launch

Amazon RDS

or

EC2 Database

Choose

Private Subnet

Notice

No Public IP.

Now

Internet

↓

Database

✗ Impossible

Step 10: Create Database Security Group

Create

Database-SG

Inbound

Instead of

Anywhere

choose

Source

↓

WebServer-SG

Meaning

Only the Web Server can connect.

Allow

3306

(MySQL)

Result

Web Server

↓

Database

Internet



Database



Step 11: Create Private NACL

Create

Private-NACL

Inbound

3306

Allow

Outbound

All

Allow

Associate it with

Private Subnet

Step 12: Create Elastic IP

Before creating NAT,

Allocate

Elastic IP

AWS gives something like

54.xx.xx.xx

This public IP belongs to NAT.

Step 13: Create NAT Gateway

Go

VPC



NAT Gateway



Create

Choose

Public Subnet

Attach

Elastic IP

Now

Internet



NAT Gateway

is possible.

Step 14: Update Private Route Table

Earlier

Private Route Table

Only Local Route

Now add

Destination

0.0.0.0/0

↓

Target

NAT Gateway

Now

Database

↓

Internet

works.

Final Traffic Flow

User Opens Website

Browser

|

|

Internet

|

Internet Gateway

|

Public Route Table

|

Public NACL

|

Web Security Group

|

EC2 Web Server

|

Private Route

|

Database Security Group

|

Database

Database Downloads Updates

```
Database
  |
Private Route Table
  |
NAT Gateway
  |
Internet Gateway
  |
Internet
```

Hacker Tries to Access Database

```
Internet
  |
Internet Gateway
  |
Private Database
```



Blocked

Because

There is **no direct Internet route** to the private subnet.

AWS Resources You Will Create (In Order)

Step	AWS Resource	Why?
1	VPC	Create a private network
2	Public Subnet	Place Internet-facing resources
3	Private Subnet	Place internal resources
4	Internet Gateway	Connect VPC to the Internet
5	Public Route Table	Send Internet traffic to the IGW
6	EC2 Web Server	Host the web application

Step	AWS Resource	Why?
7	Web Security Group	Control access to the web server
8	Public NACL	Protect the public subnet
9	Database (RDS/EC2)	Store application data
10	Database Security Group	Allow only the web server to connect
11	Private NACL	Protect the private subnet
12	Elastic IP	Give the NAT Gateway a public IP
13	NAT Gateway	Allow outbound Internet access from the private subnet
14	Private Route Table	Route outbound traffic through the NAT Gateway

Learning Tip

Since you're preparing for cloud and MLOps roles, I recommend learning this in **three phases** rather than all at once:

1. **Phase 1:** VPC, Subnets, Route Tables, and Internet Gateway.
2. **Phase 2:** Security Groups, NACLs, and NAT Gateway (understanding why each exists).
3. **Phase 3:** Build the entire architecture in the AWS Console, then recreate it using Terraform or AWS CLI.

This progression makes the concepts much easier to understand than trying to memorize the entire architecture at once.

ChatGPT can make mistakes. Check important info.